

CMP 4294 – Introduction to Artificial Intelligence

IN ACADEMIC PARTNERSHIP WITH



CREATING AI LEADERS



Customer Segmentation in E-Commerce Using Purchasing Behaviour and K-Means Clustering

Student Name: MD Sapota

Student ID: 26152255

Module Leader: Mariam Adedoyin-Olowe

5 October 2026

Table of Contents

Contents

Acknowledgement	4
Abstract	5
1 Domain Description	6
2 Problem Definition	6
3 Literature Review	6
3.1 K-Means Clustering	6
3.2 RFM Customer Behaviour	6
3.3 Cluster Validation and Critical Interpretation	7
4 Dataset Description	8
4.1 Where the Data Comes From	8
4.2 Dataset Characteristics	8
4.3 CSV Preview and Notebook Access	8
4.4 Feature Descriptions	9
5 Data Pre-Processing	9
5.1 Loading and Inspecting the Dataset	9
5.2 Cleaning the Data and Creating Transaction Value	11
5.3 Creating RFM Features	13
6 Descriptive Analysis Techniques	14
6.1 RFM Distributions	14
6.2 Why Scaling Is Necessary	14
7 Experiments – K-Means Clustering	14
7.1 Selecting the Number of Clusters	14
7.2 Elbow and Silhouette Evidence	16
7.3 Final K-Means Model and Cluster Profiles	17
7.4 Robustness to Initialisation	18
8 Analysis of Results and Conclusion	18
8.1 Cluster Results	18
8.2 Business Interpretation	20
8.3 Strengths and Limitations	20
8.4 Conclusion	20
References	21
Appendix A – Reproducibility and Submitted Files	22

Table of Figures

1	Data-quality issues in the 2,000-row project dataset. The empty missing-values panel correctly represents zero missing values; cancellation and non-positive-quantity counts overlap.	10
2	Distribution of cleaned transaction value.	12
3	RFM distributions for the 141 customers.	14
4	Elbow method: inertia against the number of clusters.	16
5	Silhouette score against the number of clusters.	16
6	Customer clusters: Frequency versus Monetary value.	19
7	Mean cluster profiles. Units are Recency in days, Frequency in distinct orders, and Monetary value in GBP.	19

Table of Tables

1	Dataset characteristics.	8
2	Preview of the submitted CSV dataset.	8
3	Feature descriptions.	9
4	RFM descriptive statistics.	13
5	Evaluation of candidate values of K	15
6	Cluster profiles for the final K-Means model ($K = 4$).	17

Acknowledgement

I would like to thank Birmingham City University, Sunway College and the module teaching team for providing the opportunity to complete this practical artificial-intelligence project. The assessment provided hands-on experience of preparing real e-commerce transaction data, applying a taught clustering method and interpreting the results critically. I also acknowledge the maintainers of the UCI Online Retail dataset and the open-source Python ecosystem, particularly pandas, NumPy, Matplotlib and scikit-learn, which made the reproducible analysis possible.

Abstract

Raw retail transaction records rarely reveal which customers behave alike. This report examines whether K-Means clustering can identify meaningful behavioural groups in e-commerce purchasing data. A reproducible 2,000-row subset of the UCI Online Retail dataset was used to build Recency, Frequency and Monetary (RFM) features for 141 customers. The features were standardised, and $K = 2$ to 8 was tested using inertia and silhouette analysis. $K = 4$ was selected (silhouette 0.4714) and was stable across twenty initialisations, producing four segments: high-value loyal, frequent regular, occasional, and lapsed or low-engagement. A key limitation is that the customer-aware subset compresses the upper Frequency and Monetary tails.

1. Domain Description

E-commerce generates transaction-level data in which each row records a product purchased within an order. Individually, these rows describe what was sold; across many customers they carry behavioural information about how often people buy, how recently they bought and how much they spend. These patterns matter because customers differ widely in value and engagement, and retailers must decide where to focus limited retention and marketing effort. Customer segmentation groups customers who behave similarly, so that decisions can be framed around a few interpretable groups rather than thousands of individual histories. This report tests one data-driven way to form such groups without assuming that segmentation by itself improves sales.

2. Problem Definition

The research question is: *can K-Means clustering identify meaningful customer groups from e-commerce purchasing behaviour so that the retailer can better understand high-value, regular, occasional and low-engagement customers?* Raw transaction records do not directly show which customers behave in similar ways. The task is descriptive rather than predictive: no outcome is forecast and no label is known in advance. The aim is to discover similar behavioural groups using one technique, K-Means clustering. Behaviour is summarised using Recency, Frequency and Monetary value, so each customer is represented by a small set of interpretable measures. Success is judged by whether the clusters are distinct, statistically supported and explainable in business terms.

3. Literature Review

3.1 K-Means Clustering

K-Means is a centroid-based clustering method introduced by MacQueen (1967) and formalised through least-squares quantisation by Lloyd (1982). It partitions observations into K groups by minimising within-cluster variance, assigning each point to the nearest centroid under Euclidean distance. Two properties are especially important here. First, distance is scale-sensitive, so variables measured in different units should be standardised before clustering (Pedregosa et al., 2011). Second, K must be chosen in advance, which makes model selection a substantive decision rather than a default.

3.2 RFM Customer Behaviour

RFM is a long-established representation in which Recency, Frequency and Monetary value summarise purchasing behaviour (Hughes, 1994; Fader, Hardie and Lee, 2005). Using the same Online Retail data family, Chen, Sain and Guo (2012) demonstrated RFM-based customer segmentation for online retail, showing the direct relevance of behavioural grouping to this domain. Market-segmentation theory similarly frames customers as heterogeneous groups that may require different treatment (Wedel and Kamakura, 2000).

3.3 Cluster Validation and Critical Interpretation

Rousseeuw (1987) proposed the silhouette coefficient to assess how well a point fits its own cluster relative to the next closest cluster, providing a separation measure alongside inertia. K-Means nevertheless assumes roughly spherical and comparable clusters and offers no causal explanation: it describes similarity only. These properties shape both the method used here and the cautious interpretation of its outputs.

4. Dataset Description

4.1 Where the Data Comes From

The source is the UCI Online Retail dataset (Chen, 2015), accessed through a Kaggle mirror (Kaggle, 2021). UCI records 541,909 transaction rows and licenses the dataset under CC BY 4.0. The project uses a reproducible 2,000-row customer-aware subset so the analysis remains manageable while retaining repeated purchasing histories.

4.2 Dataset Characteristics

Table 1. Dataset characteristics.

Attribute	Detail
Dataset name	Online Retail
Source	UCI Machine Learning Repository via Kaggle mirror
Original records	541,909 transaction rows
Original features	8
Project records	2,000 transaction rows
Project customers	141 unique CustomerID values
Data types	Identifier, text, numeric, DateTime, categorical
Main technique	K-Means clustering
Behavioural features	Recency, Frequency, Monetary value

4.3 CSV Preview and Notebook Access

Table 2 shows actual rows from the submitted `ecommerce_2000.csv`. The complete analysis is in the separately submitted Jupyter notebook, which runs sequentially in Google Colab or Jupyter with the included CSV. Repository links are omitted to preserve anonymous marking.

Table 2. Preview of the submitted CSV dataset.

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
573152	23493	VINTAGE DOILY TRAVEL SEWING KIT	6	27-10-2011 20:16	1.95	17530	United Kingdom
536595	20679	EDWARDIAN PARASOL RED	1	01-12-2010 17:24	5.95	13576	United Kingdom
548315	84945	MULTI COLOUR SILVER T-LIGHT HOLDER	12	30-03-2011 12:34	0.85	16019	United Kingdom
574086	23391	I LOVE LONDON MINI BACKPACK	4	03-11-2011 08:39	4.15	15651	United Kingdom
568151	23409	PHOTO FRAME LINEN AND LACE LARGE	6	25-09-2011 11:39	3.75	17506	United Kingdom

4.4 Feature Descriptions

Table 3. Feature descriptions.

Feature	Type	Description
InvoiceNo	Identifier	Order identifier; a C prefix marks a cancellation
StockCode	Identifier	Product code
Description	Text	Product description
Quantity	Numerical	Units purchased on the transaction line
InvoiceDate	DateTime	Date and time of the order
UnitPrice	Numerical	Price per unit in GBP
CustomerID	Identifier	Customer key used for customer-level RFM features
Country	Categorical	Customer country

5. Data Pre-Processing

5.1 Loading and Inspecting the Dataset

The notebook loads the submitted CSV and checks its shape, data types, missing values, duplicates, cancellations, quantities and prices.

Code excerpt 1: loading and initial inspection

The notebook checks for the CSV beside the notebook and then in data/. If neither copy exists, its verified Colab fallback retrieves the same dataset. The repository-specific URL is omitted here to preserve anonymity.

```
from pathlib import Path

root_csv = Path("ecommerce_2000.csv")
repo_csv = Path("data/ecommerce_2000.csv")

if root_csv.exists():
    DATA_PATH = root_csv
elif repo_csv.exists():
    DATA_PATH = repo_csv
else:
    # Colab fallback downloads the same verified project CSV
    # used for this report.
    DATA_PATH = root_csv

df = pd.read_csv(
    DATA_PATH,
    dtype={"CustomerID": "string"}
)

print("Shape:", df.shape)
print("Missing CustomerID:",
      int(df["CustomerID"].isna().sum()))
print("Duplicate rows:",
      int(df.duplicated().sum()))
print("Cancelled invoices:",
```

```

int(df["InvoiceNo"].astype(str).str.startswith("C").sum())
print("Rows with Quantity <= 0:",
      int((df["Quantity"] <= 0).sum()))
print("Rows with UnitPrice <= 0:",
      int((df["UnitPrice"] <= 0).sum()))
print("Unique customers:",
      int(df["CustomerID"].nunique()))

```

```

Shape: (2000, 8)
Missing CustomerID: 0
Duplicate rows: 0
Cancelled invoices: 185
Rows with Quantity <= 0: 185
Rows with UnitPrice <= 0: 1
Unique customers: 141

```

Missing values: 0 across all 8 attributes. The 185 cancelled invoices are the same 185 rows with non-positive quantity; the two checks overlap.

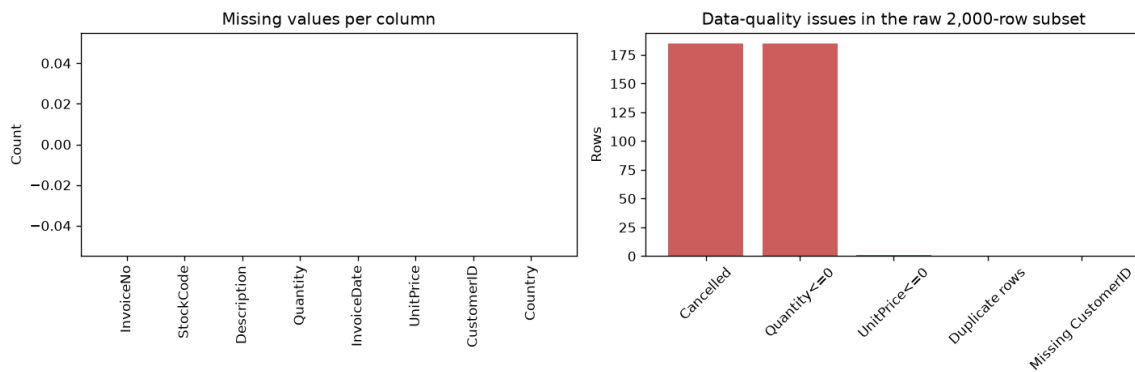


Figure 1. Data-quality issues in the 2,000-row project dataset. The empty missing-values panel correctly represents zero missing values; cancellation and non-positive-quantity counts overlap.

5.2 Cleaning the Data and Creating Transaction Value

Cancelled invoices were removed because they represent returns rather than purchases. The 185 cancellations are the same 185 rows with non-positive quantity, so the counts overlap rather than represent 370 rows. Non-positive quantities/prices and exact duplicates were excluded, and CustomerID was required. The cleaned dataset contains 1,814 rows and 141 customers.

Code excerpt 2: cleaning and TotalPrice

```
rows_start = len(df)
data = df.copy()

# Remove cancelled invoices (returns)
data = data[
    ~data["InvoiceNo"].astype(str).str.startswith("C")
]

# Keep positive purchase quantities and prices
data = data[
    (data["Quantity"] > 0) &
    (data["UnitPrice"] > 0)
]

# Prevent double-counting exact repeated rows
data = data.drop_duplicates()

# Customer key is required for segmentation
data = data.dropna(subset=["CustomerID"])

data["InvoiceDate"] = pd.to_datetime(
    data["InvoiceDate"], format="%d-%m-%Y %H:%M"
)
data["TotalPrice"] = (
    data["Quantity"] * data["UnitPrice"]
)
```

```
Initial project rows: 2000
After removing cancellations: 1815
After removing invalid quantity/price: 1814
After removing duplicates: 1814
Final cleaned rows: 1814
Unique customers after cleaning: 141
```

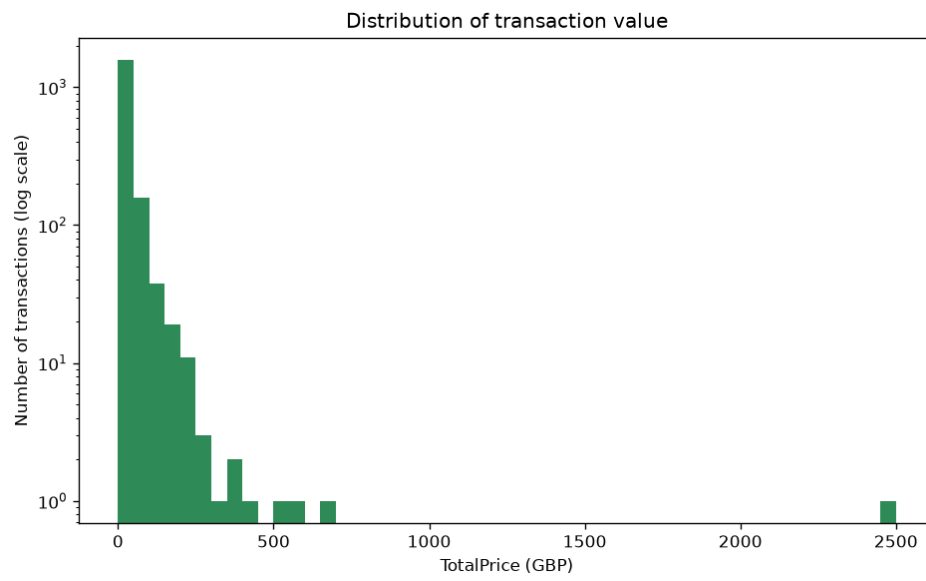


Figure 2. Distribution of cleaned transaction value.

5.3 Creating RFM Features

RFM converts line-level purchases into one row per customer. The notebook uses a reference date one day after the latest valid transaction. Recency is the number of days since the customer's latest purchase, Frequency is the number of distinct invoices, and Monetary is total cleaned spend.

Code excerpt 3: customer-level RFM features

```
snapshot = data["InvoiceDate"].max() + pd.Timedelta(days=1)

rfm = data.groupby("CustomerID").agg(
    Recency=(
        "InvoiceDate",
        lambda s: (snapshot - s.max()).days
    ),
    Frequency=("InvoiceNo", "nunique"),
    Monetary=("TotalPrice", "sum"),
)

print("Snapshot date:", snapshot.date())
print("Customers:", len(rfm))
rfm.describe().round(2)
```

Table 4. RFM descriptive statistics.

Statistic	Recency (days)	Frequency (orders)	Monetary (GBP)
Mean	73.08	4.51	366.06
Median	33.00	4.00	215.55

Three sampling constraints must be acknowledged: at most three line items per order, at most ten orders per customer, and exclusion of customers with more than thirty source orders. These choices preserve repeated histories under the 2,000-row project scope but compress the upper Frequency and Monetary tails.

6. Descriptive Analysis Techniques

6.1 RFM Distributions

Figure 3 shows that Recency and Monetary are strongly right-skewed. Most customers have relatively recent activity and modest spend, while a small group of customers account for much larger monetary values. Frequency is bounded by the sampling design, which explains the visible upper limit at ten orders. These distributions justify scaling before Euclidean-distance clustering.

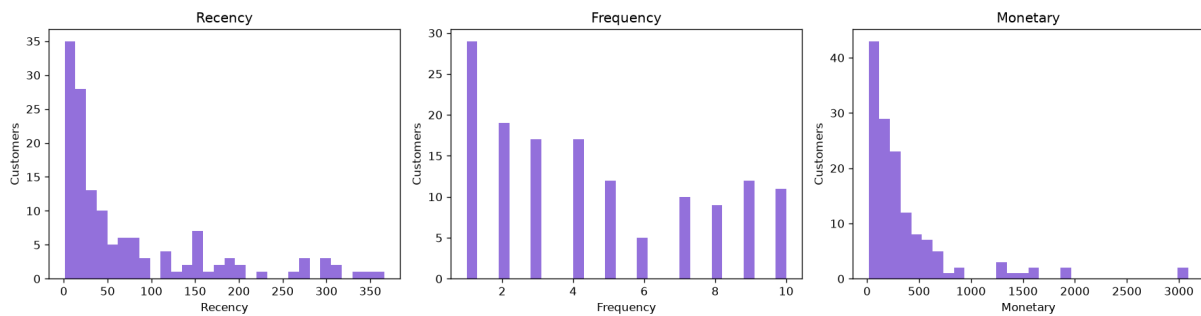


Figure 3. RFM distributions for the 141 customers.

6.2 Why Scaling Is Necessary

Recency is measured in days, Frequency in orders and Monetary in pounds. Without standardisation, Monetary would dominate Euclidean distance simply because it has the largest numerical scale. The submitted notebook therefore transforms all three RFM features to comparable standardised values.

Code excerpt 4: feature scaling

```
scaler = StandardScaler()

X = scaler.fit_transform(
    rfm[["Recency", "Frequency", "Monetary"]].values
)

print("Scaled means:",
      np.round(X.mean(axis=0), 3))
print("Scaled stds :",
      np.round(X.std(axis=0), 3))
```

7. Experiments – K-Means Clustering

7.1 Selecting the Number of Clusters

The number of clusters was not chosen arbitrarily. Candidate solutions from $K = 2$ through 8 were fitted with a fixed random seed and `n_init=10`. Inertia and silhouette score were recorded for each candidate.

Code excerpt 5: testing K from 2 to 8

```
k_values = list(range(2, 9))
```

```

inertias = []
silhouettes = []

for k in k_values:
    model = KMeans(
        n_clusters=k,
        random_state=RANDOM_STATE,
        n_init=10
    )
    labels = model.fit_predict(X)
    inertias.append(model.inertia_)
    silhouettes.append(
        silhouette_score(X, labels)
    )

results = pd.DataFrame({
    "K": k_values,
    "Inertia": np.round(inertias, 2),
    "Silhouette": np.round(silhouettes, 4)
})
results

```

Table 5. Evaluation of candidate values of K.

K	Inertia	Silhouette
2	247.11	0.4103
3	157.79	0.4329
4	91.01	0.4714
5	73.21	0.4625
6	56.88	0.4693
7	43.30	0.4270
8	38.53	0.3896

7.2 Elbow and Silhouette Evidence

Inertia falls sharply up to $K = 4$ and then flattens, while the silhouette score reaches its maximum at $K = 4$ (0.4714). The values for $K = 4, 5$ and 6 are relatively close, so $K = 4$ should be treated as the best-supported modelling choice for this sample rather than an absolute natural truth.

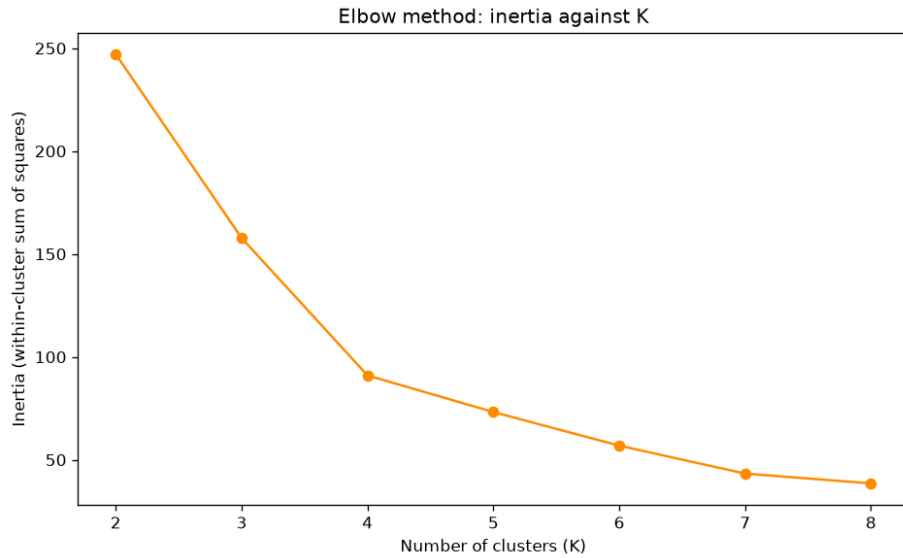


Figure 4. Elbow method: inertia against the number of clusters.

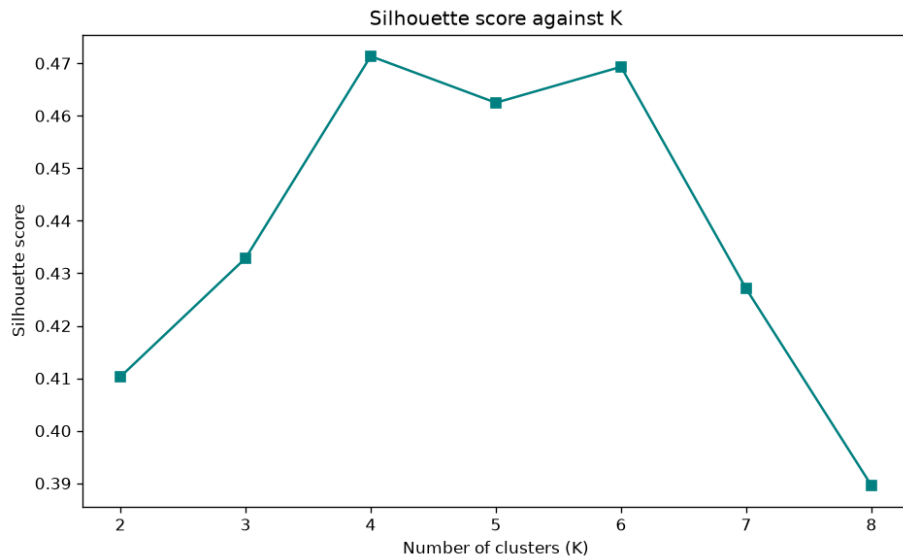


Figure 5. Silhouette score against the number of clusters.

7.3 Final K-Means Model and Cluster Profiles

The final model uses four clusters, fixed `random_state=42` and `n_init=10`. Cluster profile statistics are then calculated directly from the customer-level RFM table.

Code excerpt 6: final model and profiling

```
final_k = 4

kmeans = KMeans(
    n_clusters=final_k,
    random_state=RANDOM_STATE,
    n_init=10
)
rfm["Cluster"] = kmeans.fit_predict(X)

print("Final inertia:",
      round(float(kmeans.inertia_), 2))
print("Final silhouette:",
      round(float(
          silhouette_score(X, rfm["Cluster"])
      ), 4))

profile = rfm.groupby("Cluster").agg(
    Customers=("Recency", "size"),
    Recency_mean=("Recency", "mean"),
    Recency_median=("Recency", "median"),
    Frequency_mean=("Frequency", "mean"),
    Frequency_median=("Frequency", "median"),
    Monetary_mean=("Monetary", "mean"),
    Monetary_median=("Monetary", "median"),
).round(2)
profile
```

Table 6. Cluster profiles for the final K-Means model ($K = 4$).

Cluster	Customers	Recency	Frequency	Monetary (GBP)	Interpretation
0	11	26.55	8.45	1,811.52	High-value loyal
2	40	24.02	7.92	415.52	Frequent regular
1	60	38.98	2.87	187.98	Occasional
3	30	223.73	1.80	126.28	Lapsed / low-engagement

7.4 Robustness to Initialisation

Because K-Means can depend on initial centroid placement, the selected $K = 4$ solution was re-fitted across random-state values 0–19 with `n_init=10`. The silhouette score was 0.4714 in every run, the mean Adjusted Rand Index against the reference solution was 1.0, and only one cluster-size signature appeared (11, 30, 40, 60). This indicates stable initialisation for this project dataset, although it does not prove stability across different samples.

Code excerpt 7: stability check

```
reference = rfm["Cluster"].values
robust_sil = []
robust_ari = []

for seed in range(20):
    labels_seed = KMeans(
        n_clusters=final_k,
        random_state=seed,
        n_init=10
    ).fit_predict(X)

    robust_sil.append(
        silhouette_score(X, labels_seed)
    )
    robust_ari.append(
        adjusted_rand_score(reference, labels_seed)
    )

print("Silhouette mean:",
      round(np.mean(robust_sil), 4))
print("ARI mean:",
      round(np.mean(robust_ari), 4))
```

```
Silhouette min / mean / max: 0.4714 / 0.4714 / 0.4714
Mean Adjusted Rand Index vs seed 42: 1.0000
Distinct cluster-size signatures: 1
```

8. Analysis of Results and Conclusion

8.1 Cluster Results

The four clusters differ clearly in the measured behaviour. Cluster 0 contains eleven customers who bought recently (mean Recency 26.55 days), most often (mean Frequency 8.45 orders) and with by far the highest spend (mean Monetary £1,811.52); this is best described as a high-value loyal group. Cluster 2 (forty customers) is also recent and frequent (Recency 24.02; Frequency 7.92) but with moderate spend (£415.52), suggesting a frequent regular group. Cluster 1 is larger (sixty customers) but purchases far less often (Frequency 2.87) and spends little (£187.98), indicating an occasional group. Cluster 3 (thirty customers) is least recent (Recency 223.73 days), least frequent (1.80 orders) and lowest spend (£126.28), consistent with a lapsed or low-engagement group. The numeric identifiers are arbitrary; the business labels are interpretations of the computed profiles

rather than ground truth.

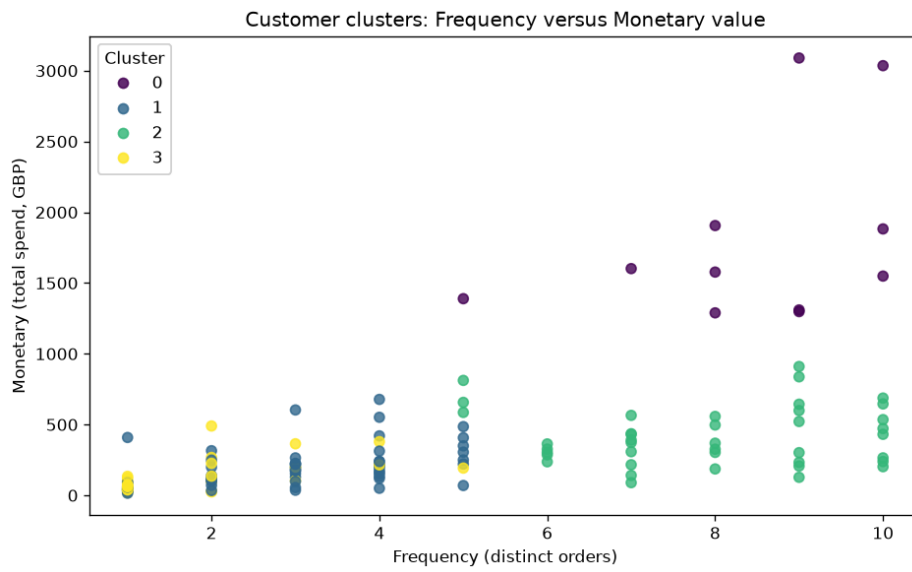


Figure 6. Customer clusters: Frequency versus Monetary value.

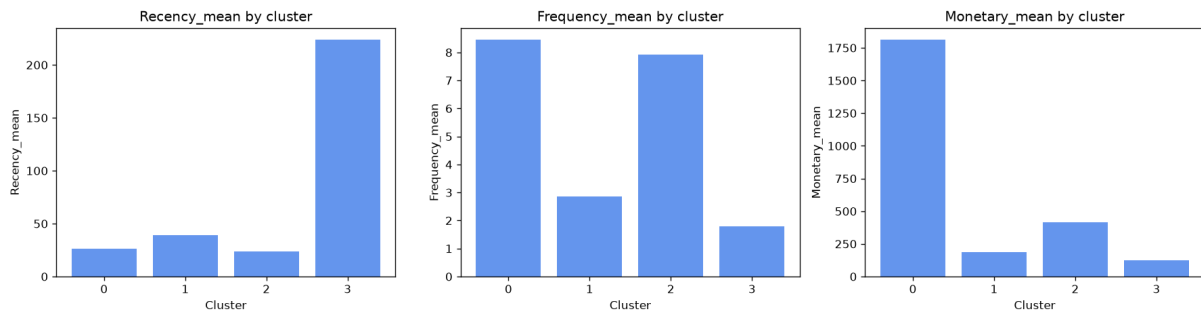


Figure 7. Mean cluster profiles. Units are Recency in days, Frequency in distinct orders, and Monetary value in GBP.

8.2 Business Interpretation

Business implications follow from the observed profiles and are stated as possibilities rather than proven campaign effects. Retention or loyalty attention may be appropriate for the high-value loyal group, given its recent and frequent behaviour. The frequent regular group buys often at lower spend, so there may be potential to increase basket value. The occasional group is numerous but infrequent, suggesting an opportunity to encourage repeat purchases. The lapsed group could be a candidate for reactivation campaigns. These are hypotheses for later testing; the clustering data do not measure whether any intervention would increase revenue.

8.3 Strengths and Limitations

The analysis is reproducible and uses one taught method throughout. K was selected using two complementary measures, the final clusters were profiled with interpretable customer statistics, and the robustness test shows the chosen solution is not sensitive to the tested initialisations. However, customer-aware sampling changes the original distribution. Capping line items and orders compresses the upper Frequency and Monetary tails, so the 141 customers cannot automatically represent the full retailer population. RFM also ignores product mix, margin, acquisition source, marketing exposure and website activity. K-Means assumes roughly spherical clusters, and K selection remains a modelling decision. Most importantly, clustering identifies similarity, not causation.

8.4 Conclusion

The research question is supported for this project sample. K-Means identified four meaningful behavioural groupings from standardised RFM features: high-value loyal, frequent regular, occasional, and lapsed or low-engagement customers. $K = 4$ produced the highest silhouette score (0.4714), aligned with the inertia elbow and remained stable across twenty tested initialisations. The segments offer a useful descriptive view of purchasing behaviour, but the sampling design and RFM limitations mean that the results should be interpreted cautiously rather than generalised automatically to the complete Online Retail population.

References

- Chen, D. (2015) *Online Retail*. [dataset] UCI Machine Learning Repository. Available at: <https://doi.org/10.24432/C5BW33> [Accessed 5 October 2026].
- Chen, D., Sain, S.L. and Guo, K. (2012) Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining. *Journal of Database Marketing & Customer Strategy Management*, 19(3), pp. 197–208. <https://doi.org/10.1057/dbm.2012.17>.
- Fader, P.S., Hardie, B.G.S. and Lee, K.L. (2005) RFM and CLV: Using iso-value curves for customer base analysis. *Journal of Marketing Research*, 42(4), pp. 415–430. <https://doi.org/10.1509/jmkr.2005.42.4.415>.
- Hughes, A.M. (1994) *Strategic Database Marketing*. Chicago: Probus Publishing.
- Kaggle (2021) *Online Retail Business*. [dataset] Available at: <https://www.kaggle.com/datasets/umerkk12/online-retail-business> [Accessed 5 October 2026].
- Lloyd, S.P. (1982) Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), pp. 129–137. <https://doi.org/10.1109/TIT.1982.1056489>.
- MacQueen, J. (1967) Some methods for classification and analysis of multivariate observations. In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1*. Berkeley: University of California Press, pp. 281–297.
- Pedregosa, F. et al. (2011) Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Rousseeuw, P.J. (1987) Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, pp. 53–65. [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7).
- Wedel, M. and Kamakura, W.A. (2000) *Market Segmentation: Conceptual and Methodological Foundations*. 2nd edn. Boston: Kluwer Academic Publishers.

Appendix A – Reproducibility and Submitted Files

The submission package contains the Jupyter notebook and the exact CSV analysed in this report:

- Submitted Jupyter Notebook (.ipynb)
- ecommerce_2000.csv

The notebook contains all Python used for loading, cleaning, RFM construction, scaling, K selection, K-Means fitting, cluster profiling, robustness testing and figure generation. It is designed to run sequentially in Google Colab or Jupyter without requiring Kaggle credentials. The 2,000-row project dataset was generated reproducibly from the full source file and is included with the submission.